Recursion and Backtracking

Practice Problems 1, 2 and 3 – Solutions

Problem 1: Understanding Recursion Mechanics

a) Recursive design (pseudocode)

```
function totalSize(node):
    if node is a FILE:                 // BASE CASE (leaf node)
        return node.size
    total = 0                          // node is a DIRECTORY
    for each child in node.children:   // RECURSIVE CASE
        total = total + totalSize(child)
    return total                       // empty directory returns 0
```

- **Base case:** the node is a file, so return its own size. Files are the leaves of the tree, so recursion always ends there.
- **Recursive case:** the node is a directory. Its size is the sum of the sizes of its children, and each child is a smaller sub-problem (a smaller subtree).

b) Trace on the given structure

Call: totalSize(project). Indentation shows the depth of the call stack.

```
totalSize(project)                          stack: [project]
  totalSize(src)                            stack: [project, src]
    totalSize(main.java)   -> 100           (base case)
    totalSize(utils.java)  -> 50            (base case)
  src returns 100 + 50 = 150
  totalSize(docs)                           stack: [project, docs]
    totalSize(readme.txt)  -> 10            (base case)
    totalSize(guides)                       stack: [project, docs, guides]
      totalSize(setup.pdf) -> 200           (base case; depth 4, max)
    guides returns 200
  docs returns 10 + 200 = 210
  totalSize(config.xml)    -> 20            (base case)
project returns 150 + 210 + 20 = 380
```

Call	Returns	Stack depth	Values combined
main.java	100	3 (project, src, main)	–
utils.java	50	3	–
src	150	2	100 + 50
readme.txt	10	3	–
setup.pdf	200	4 (project, docs, guides, setup)	–
guides	200	3	200
docs	210	2	10 + 200
config.xml	20	2	–
project	380 KB	1	150 + 210 + 20

Total size = 380 KB. The call stack grows while we go down into sub-directories and unwinds as each call returns its value to its parent.

c) Time complexity

- Every node (file or directory) is visited exactly once, and each visit does constant work plus one addition per child. So **T(n) = O(F + D) = O(n)**, where F = number of files, D = number of directories and n = F + D.
- For the given tree: 5 files + 4 directories = 9 nodes, so 9 calls in total.
- Directories matter too, not only files: a tree with many empty directories still takes time proportional to the number of directories.
- **Space:** O(h), where h is the height of the tree (recursion stack). Here h = 4. Worst case (a single deeply nested chain) it becomes O(n).

d) Iterative solution

Yes. Recursion can always be replaced by an explicit **stack** (DFS); a queue (BFS) also works because the order of visiting does not matter for a sum.

```
function totalSizeIter(root):
    stack = [root]; total = 0
    while stack is not empty:
        node = stack.pop()
        if node is FILE: total += node.size
        else: for each child in node.children: stack.push(child)
    return total
```

	Recursive	Iterative (explicit stack)
Code	Short, mirrors the tree structure	Longer, manual stack handling
Time	O(n)	O(n)
Space	O(h) call stack	O(max frontier), up to O(n) for a very wide directory
Risk	StackOverflow for very deep trees	No stack overflow; stack lives on the heap
Per-directory subtotals	Natural (return value of each call)	Harder, needs post-order bookkeeping

e) Symbolic link creating a cycle

If a symlink points back to an ancestor (for example docs/link -> project), the recursion would go project -> docs -> link -> project -> ... forever, and we get infinite recursion / StackOverflowError (and an infinite size).

Modified base cases:

```
function totalSize(node, visited):
    if node is FILE: return node.size                    // base case 1
    realPath = canonicalPath(node)    // resolves symlinks / inode id
    if realPath in visited: return 0  // base case 2: already counted, CYCLE
    visited.add(realPath)
    total = 0
    for each child in node.children:
        total += totalSize(child, visited)
    return total
```

Alternative: simply do not follow symbolic links (count only the link file itself), or add a maximum-depth limit as a safety net. With the visited set every real directory is processed once, so time stays O(n) with O(n) extra space for the set.

Problem 2: Backtracking Template Application

Grid (4 x 4), target word **DREAM**. Rows and columns are numbered from 0.

```
      col0 col1 col2 col3
row0   C    A    T    S
row1   O    R    E    A
row2   D    E    A    M
row3   E    L    L    S
```

a) Backtracking algorithm

- **State:** (r, c, i): current cell (r, c) and index i of the letter of the word we must match next; plus the set of visited cells (current path).
- **Choices at each step:** the 8 neighbouring cells (horizontal, vertical, diagonal).
- **Constraints:** the cell is inside the grid, is not already used in the current path, and its letter equals word[i].
- **Goal condition:** i == L (length of word), meaning all letters have been matched.

```
function exist(grid, word):
    for every cell (r, c) in grid:
        if dfs(r, c, 0): return true
    return false

function dfs(r, c, i):
    if i == L: return true                          // GOAL
    if (r,c) out of bounds or visited[r][c]
       or grid[r][c] != word[i]: return false       // CONSTRAINT FAILS (prune)
    visited[r][c] = true                            // CHOOSE
    for each (dr, dc) in 8 directions:              // EXPLORE
        if dfs(r+dr, c+dc, i+1): return true
    visited[r][c] = false                           // UN-CHOOSE (backtrack)
    return false
```

Direction order used in the trace: (-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1).

b) Decision tree for "DREAM"

```
START: scan grid for D
  |
  D(2,0)                      only one D in the grid
  |
  R(1,1)                      only R adjacent to D(2,0)
  |-- E(1,2)
  |     |-- A(0,1)  -> M?  no neighbour is M          X  BACKTRACK
  |     |-- A(1,3)  -> M(2,3)                        OK SUCCESS (DFS stops here)
  |     \-- A(2,2)  -> M(2,3)                        OK valid path (not reached by
DFS)
  \-- E(2,1)
        \-- A(2,2)  -> M(2,3)                        OK valid path (not reached by
DFS)
```

Successful path found first: **D(2,0) -> R(1,1) -> E(1,2) -> A(1,3) -> M(2,3)**. The only backtracking point is A(0,1), because none of its neighbours is M. All other start cells fail at the first letter (they are not D), which prunes the search immediately.

c) Execution trace

#	Action	Path (state)	Visited change
1	Scan row-major: C,A,T,S,O,R,E,A are not D, so dfs fails at once. Next cell (2,0) = D matches word[0].	[ ]	–

#	Action	Path (state)	Visited change
2	CHOOSE D(2,0), i = 0	[D(2,0)]	mark (2,0)
3	Need R (i=1): (1,0)=O no, (1,1)=R yes. CHOOSE	[D, R(1,1)]	mark (1,1)
4	Need E (i=2): (0,0)C, (0,1)A, (0,2)T, (1,0)O no; (1,2)=E yes. CHOOSE	[D, R, E(1,2)]	mark (1,2)
5	Need A (i=3): (0,1)=A yes. CHOOSE	[D, R, E, A(0,1)]	mark (0,1)
6	Need M (i=4): neighbours C,T,O,R(used),E(used): no M. All choices fail, so BACKTRACK. State restored: pop A(0,1)	[D, R, E(1,2)]	unmark (0,1)
7	Back at E(1,2), next choices for A: (0,2)T no, (0,3)S no, (1,1)R used, (1,3)=A yes. CHOOSE	[D, R, E, A(1,3)]	mark (1,3)
8	Need M (i=4): (0,2)T, (0,3)S, (1,2)E used, (2,2)A no; (2,3)=M yes. CHOOSE, i becomes 5 = L	[D, R, E, A, M(2,3)]	mark (2,3)
9	GOAL reached, return true all the way up	DREAM found	–

When choices are made: steps 2, 3, 4, 5, 7, 8. **When backtracking occurs:** step 6. **How state is restored:** visited[r][c] is set back to false and the cell is removed from the path before returning, so the next branch sees exactly the state it had before the failed choice.

d) Worst-case time complexity

- Start cells: N x M. From each, a path of length L is explored. The first step has 8 choices; each later step has at most 7 (the previous cell is already used).
- **Worst case: O(N x M x 8^L)** (more precisely N x M x 8 x 7^(L-1)). Since a path cannot be longer than the number of cells, L <= N x M.
- Pruning makes it much faster in practice: a branch dies as soon as a letter does not match, whereas brute force would try all paths of length L without checking letters.
- **Space:** O(L) recursion depth (plus O(N x M) for the visited matrix, or O(1) extra if cells are marked in place).

e) Finding ALL ways to form the word

- **Do not stop at the first success.** Instead of "return true" at the goal, save a copy of the current path in a result list (count++) and return, so the loop keeps trying the other choices.
- **Extra data structures:** a **path list / stack** of coordinates (current path) and a **result list** (list of paths). The visited matrix is still needed.

```
function dfsAll(r, c, i):
    if i == L: results.add(copy(path)); return     // record, do not stop
    if out of bounds or visited[r][c] or grid[r][c] != word[i]: return
    visited[r][c] = true; path.push((r,c))
    for each of 8 directions: dfsAll(r+dr, c+dc, i+1)
    path.pop(); visited[r][c] = false              // backtrack
```

For "DREAM" this returns **3 paths**: D(2,0)-R(1,1)-E(1,2)-A(1,3)-M(2,3), D(2,0)-R(1,1)-E(1,2)-A(2,2)-M(2,3) and D(2,0)-R(1,1)-E(2,1)-A(2,2)-M(2,3). Worst-case time is the same order, O(N x M x 8^L), but there is no early exit. For many words, a **Trie** of the dictionary lets one DFS search all words together.

Problem 3: N-Queens Optimization and Variants

a) Search space for 8 x 8

Approach	Search space size	Value
Brute force: choose any 8 of the 64 squares	C(64, 8)	4,426,165,368
"One queen per row": 8 choices in each of 8 rows	8^8	16,777,216
(Bonus) one per row and per column (permutations)	8!	40,320

Backtracking estimate: at each row, only columns not attacked are tried, and a whole subtree is cut as soon as a row has no safe square. Counting the nodes of the pruned tree gives about **2,057 nodes** (2,056 queen placements + the empty root) to enumerate all 92 solutions, and only about **113 placements** to reach the first solution. That is roughly 0.01% of the 16.7 million "one queen per row" space, and about 0.00005% of the C(64,8) space.

Reason: the number of safe columns shrinks quickly with each row (every placed queen removes a column and up to two diagonals), so most branches die within 3 to 5 rows.

b) Optimized solution: 1D array + diagonal tracking

- board[row] = col stores the column of the queen in each row (1D array of size N instead of an N x N matrix).
- cols[c], d1[r - c + N - 1] (main diagonals, r - c is constant) and d2[r + c] (anti-diagonals, r + c is constant) are boolean arrays. A square is safe if none of its three flags is set.

```
function solveNQueens(N):
    board[0..N-1]; cols[0..N-1] = false
    d1[0..2N-2] = false; d2[0..2N-2] = false
    return place(0)

function place(row):
    if row == N: return true                          // GOAL: all placed
    for col in 0..N-1:
        if !cols[col] and !d1[row-col+N-1] and !d2[row+col]:   // O(1) check
            board[row] = col                          // CHOOSE
            cols[col] = d1[row-col+N-1] = d2[row+col] = true
            if place(row+1): return true              // EXPLORE
            cols[col] = d1[row-col+N-1] = d2[row+col] = false   // UN-CHOOSE
    return false                                      // dead end, BACKTRACK
```

Improvement over the basic solution: the basic version checks safety by scanning all previously placed queens (O(N) per check) and often uses an N x N board. With the flag arrays the check is **O(1)**, memory is **O(N)** instead of O(N^2), and one queen per row is guaranteed by construction (rows are never revisited).

c) Trace for N = 4 (first valid solution)

Diagonal indices: d1 = r - c + 3, d2 = r + c.

#	Action	board[]	cols	d1 used	d2 used
1	Row 0: place (0,0)	[0,-,-,-]	{0}	{3}	{0}
2	Row 1: (1,0) col used, (1,1) d1=3 used, place (1,2)	[0,2,-,-]	{0,2}	{3,2}	{0,3}
3	Row 2: (2,1) d2=3 used, (2,3) d1=2 used, cols 0 and 2 used. No square: BACKTRACK, remove (1,2)	[0,-,-,-]	{0}	{3}	{0}
4	Row 1: place (1,3)	[0,3,-,-]	{0,3}	{3,1}	{0,4}
5	Row 2: (2,0) col used, place (2,1)	[0,3,1,-]	{0,3,1}	{3,1,4}	{0,4,3}

#	Action	board[]	cols	d1 used	d2 used
6	Row 3: (3,2) d1=4 used, other cols used. BACKTRACK, remove (2,1)	[0,3,-,-]	{0,3}	{3,1}	{0,4}
7	Row 2: (2,2) d1=3 used, (2,3) col used. Row exhausted: BACKTRACK, remove (1,3); row 1 exhausted, remove (0,0)	[-,-,-,-]	{}	{}	{}
8	Row 0: place (0,1)	[1,-,-,-]	{1}	{2}	{1}
9	Row 1: (1,0) d2=1 used, (1,1) col used, (1,2) d1=2 used, place (1,3)	[1,3,-,-]	{1,3}	{2,1}	{1,4}
10	Row 2: place (2,0)	[1,3,0,-]	{1,3,0}	{2,1,5}	{1,4,2}
11	Row 3: (3,0),(3,1) cols used, place (3,2). row == N, SOLUTION	[1,3,0,2]	{1,3,0,2}	{2,1,5,4}	{1,4,2,5}

Backtracking points: steps 3, 6 and 7. **First valid solution:** queens at (0,1), (1,3), (2,0), (3,2):

```
. Q . .
. . . Q
Q . . .
. . Q .
```

(Statistics: 8 queen placements and 2 dead-end rows were needed; the full search for N = 4 finds 2 solutions in total.)

d) Finding just ONE solution as fast as possible

- **Stop at the first solution:** make place() return true and propagate it up (as in the pseudocode above); do not continue to collect all solutions. For N = 8 this needs about 113 placements instead of about 2,056.
- **Skip the symmetry bookkeeping** that is only needed to count/enumerate all solutions (no need to reflect or rotate solutions).
- **Use bitmasks** for cols, d1, d2 (one integer each) so the safe squares of a row are found with bit operations: avail = ~(cols | d1 | d2) & mask.
- **Better ordering / heuristics:** try the most constrained row first, or order columns from the middle; for very large N use a constructive formula (a valid solution exists for every N >= 4 and can be built in O(N)) or the min-conflicts heuristic, which is near-linear in practice.

e) N-Queens with forbidden squares

- Keep a boolean matrix forbidden[N][N] (O(N^2) space, O(1) lookup).
- In place(row), add one more condition: skip a column if forbidden[row][col] is true.

```
for col in 0..N-1:
    if forbidden[row][col]: continue                  // new constraint
    if !cols[col] and !d1[row-col+N-1] and !d2[row+col]:
        ... choose / explore / un-choose as before ...
```

- **Complexity impact:** the check is still O(1) and the worst case stays exponential (bounded by O(N!)), but forbidden squares only remove choices, so the tree becomes smaller (more pruning). The trade-off: a solution may no longer exist (for example a row where every square is forbidden), so the algorithm can return "no solution" after exploring the whole pruned tree. A quick pre-check (if any row is completely forbidden, answer "no solution" immediately) saves time.
- The classic shortcuts that assume a symmetric empty board (for example the constructive formula or only trying half of row 0) no longer apply, because the forbidden squares break the symmetry.